

瑞萨 RA8 Helium 技术培训 实验环节



瑞萨电子

2024 年 3 月

RA8 Helium intrinsic 技术培训实验

实验指南

概述

本实验将为您介绍 CPKCOR-RA8D1 的使用方法，主要涉及如何使用 J-Link OB CDC 进行 printf 调试输出，Helium intrinsic 技术的编程示例和调试。

实验目标 • 掌握 RA8 基本调试 • 掌握 RA8 Helium intrinsic 编程技术	实验材料 • CPKCOR-RA8D1 开发板及其配件 • FSP 5.1.0 • e2 studio 2023-10 及以上 • Tera Term
技能要求 • 熟悉 Renesas e2 studio 操作 • 了解 RA MCU 的外围接口 • 了解相关操作	时间 • 60 分钟

实验部分

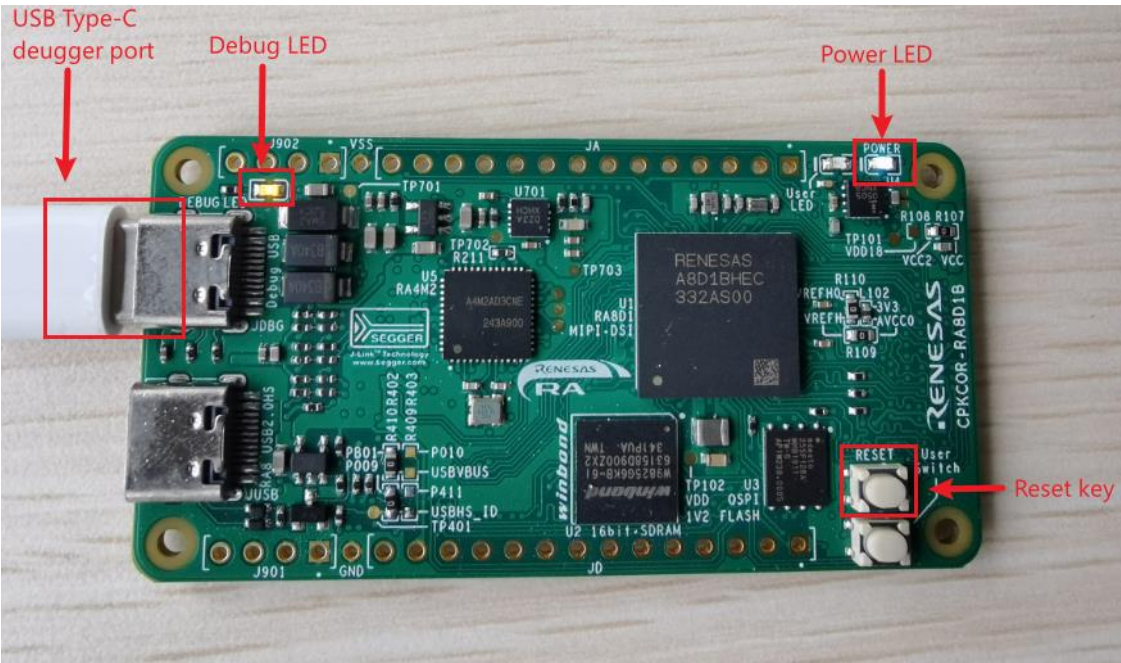
1	安装和准备	2
2	创建 printf 工程.....	7
3	Helium intrinsic 程序调试.....	19

1 安装和准备

本节要点

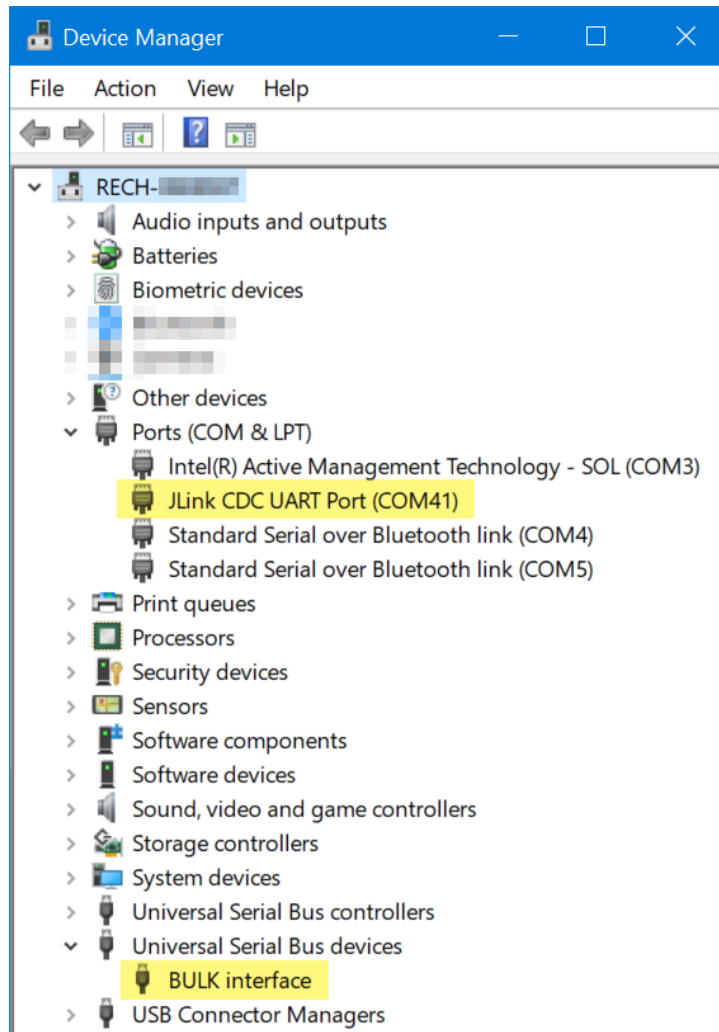
本节学习如何搭建实验以及如何将套件连接至电脑端。

步骤

1.1	首先确保已安装所有实验工具。 e2studio 2023-10 及以上版本 setup_fsp_v5_1_0_e2s_v2023-10.exe(此链接包含了 e2studio + FSP+LLVM) ➤ FSP 5.1.0 及以上版本 https://github.com/renesas/fsp/releases/download/v5.1.0/FSP_Packs_v5.1.0.exe ➤ LLVM Embedded Toolchain for Arm 17.01 https://github.com/ARM-software/LLVM-embedded-toolchain-for-Arm/releases/download/release-17.0.1/LLVMEEmbeddedToolchainForArm-17.0.1-Windows-x86_64.zip ➤ Renesas.RA_board_ra8d1_cpkcor.5.1.0.pack ➤ Tera term https://sourceforge.net/projects/tera-term/
1.2	本实验将用到如下组件：CPKCOR-RA8D1 板，USB-TypeC 数据线 1 根。
1.3	请按以下步骤操作： • 用 USB Type-C 线，一端接入 CPKCOR-RA8D1 板 USB Type-C debugger port，另一端接入 PC USB 接口。 • 左上黄色 Debug LED 和右上白色 Power LED 会常亮。 

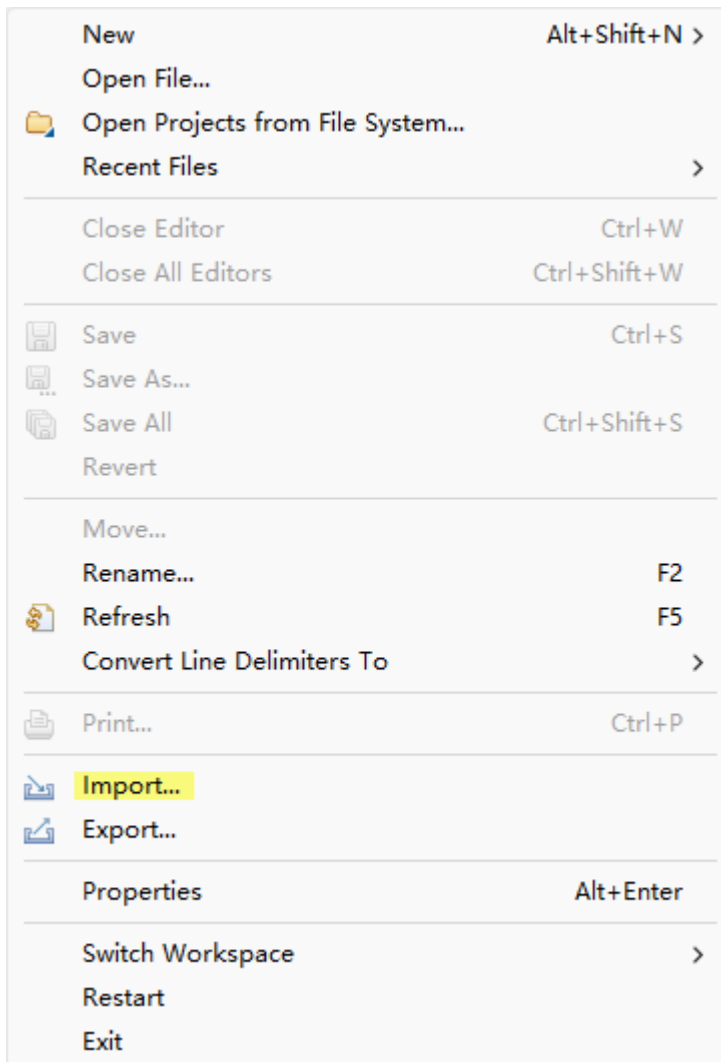
1.4

打开 PC 设备管理器，在端口(COM 和 LPT)中会显示“Jlink CDC UART Port(COMxx)”，表示 PC 已经识别到 CPKCOR-RA8D1 板上的 JLink CDC UART。在通用总线设备中会显示“BULK interface”。



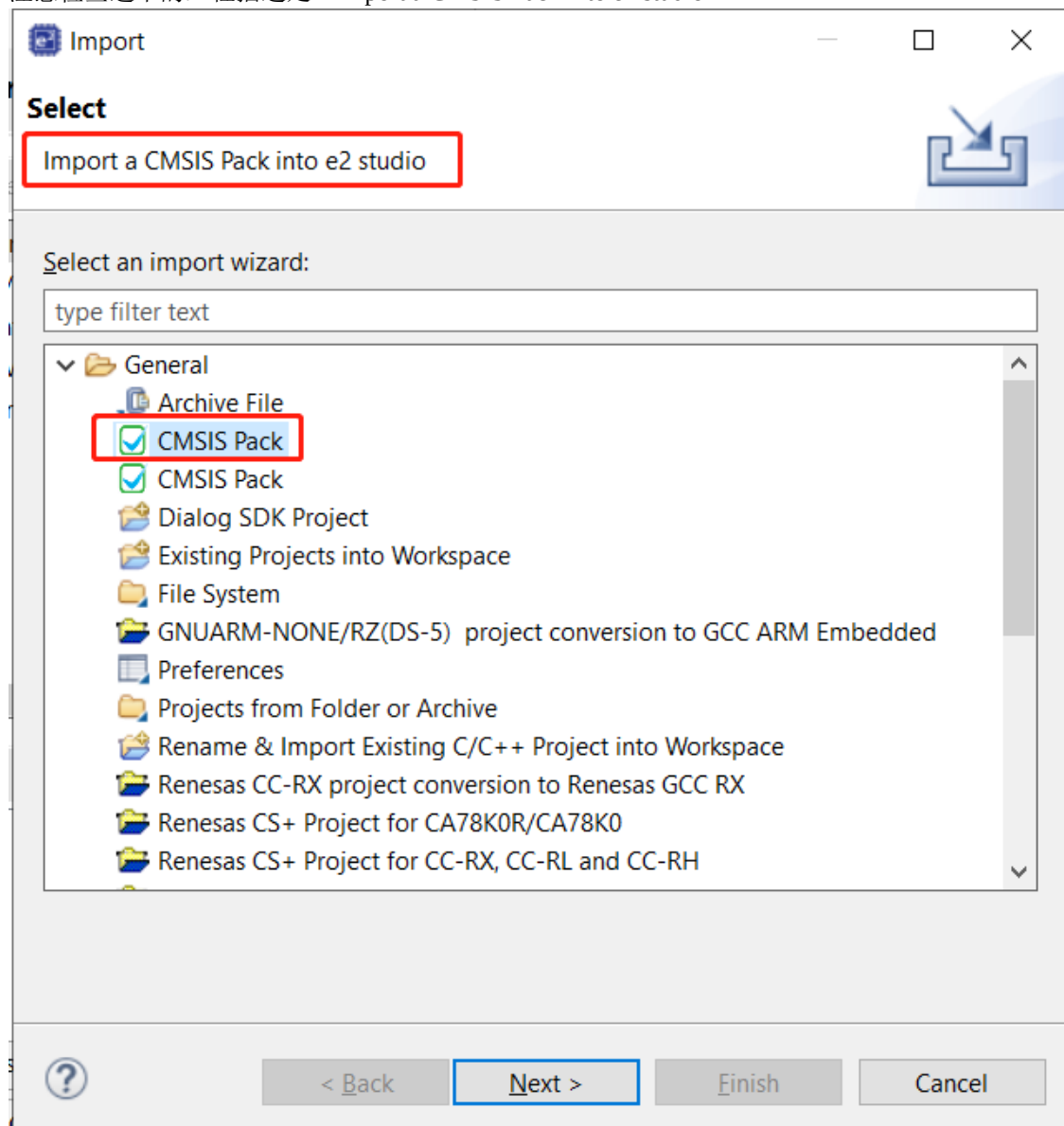
1.5

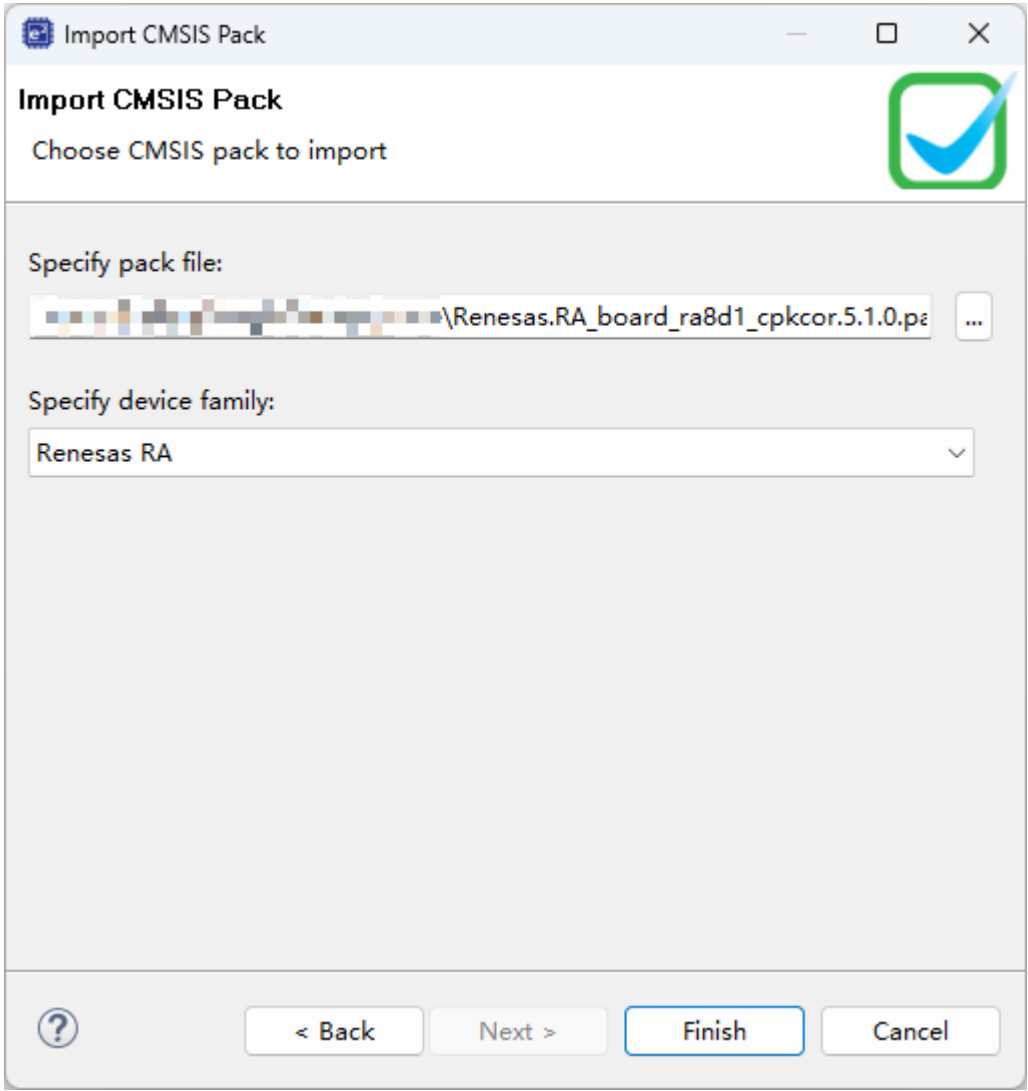
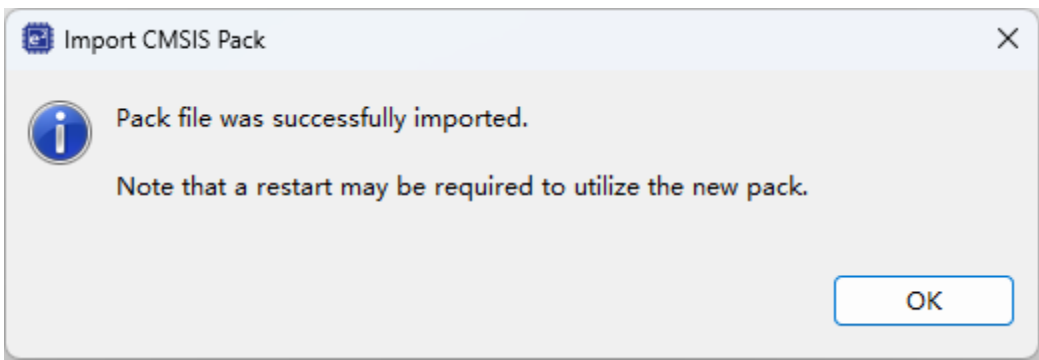
打开 e2studio 菜单中的“File” → “import...”。



1.6

在弹出的“Import”对话框中，选择“General” → “CMSIS Pack”，点击“Next”。
注意检查选中的工程描述是“Import a CMSIS Pack into e2 studio”。



1.7	在弹出的“Import CMSIS Pack”对话框中，点击“Specify pack file”右侧“...”，指定需要添加的 Renesas.RA_board_ra8d1_cpkcor.5.1.0.pack 文件。 
1.8	导入完成后，点击“OK”。 

本节完

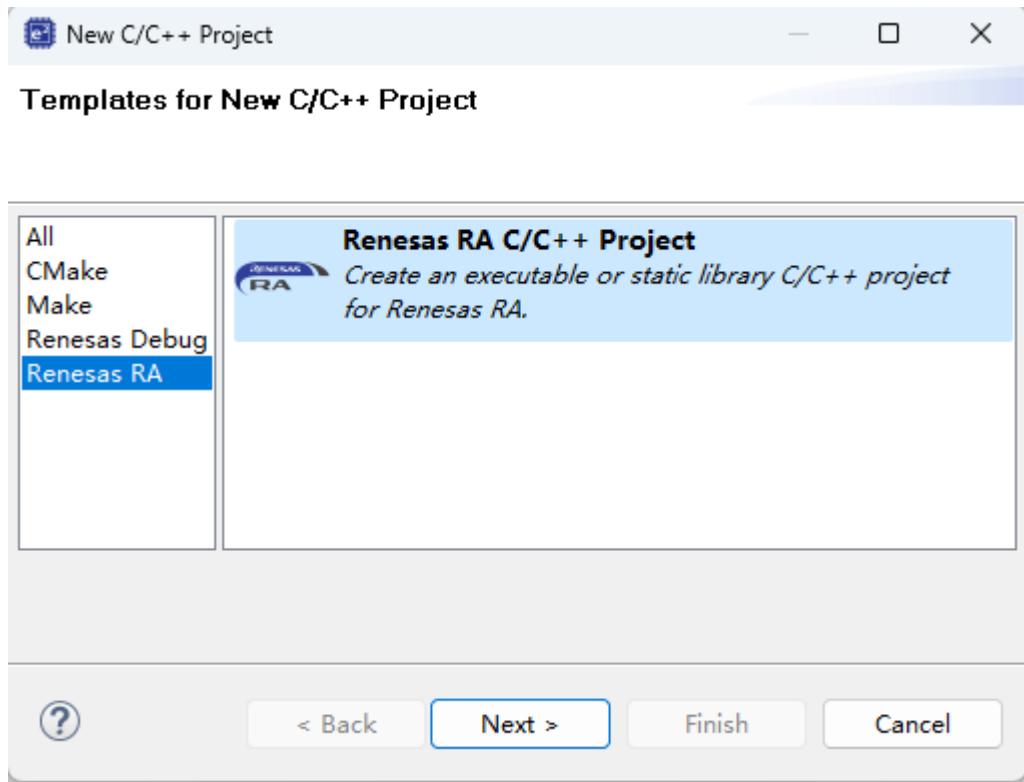
2 创建 printf 工程

本节要点

本节学习，创建一个 printf 工程。

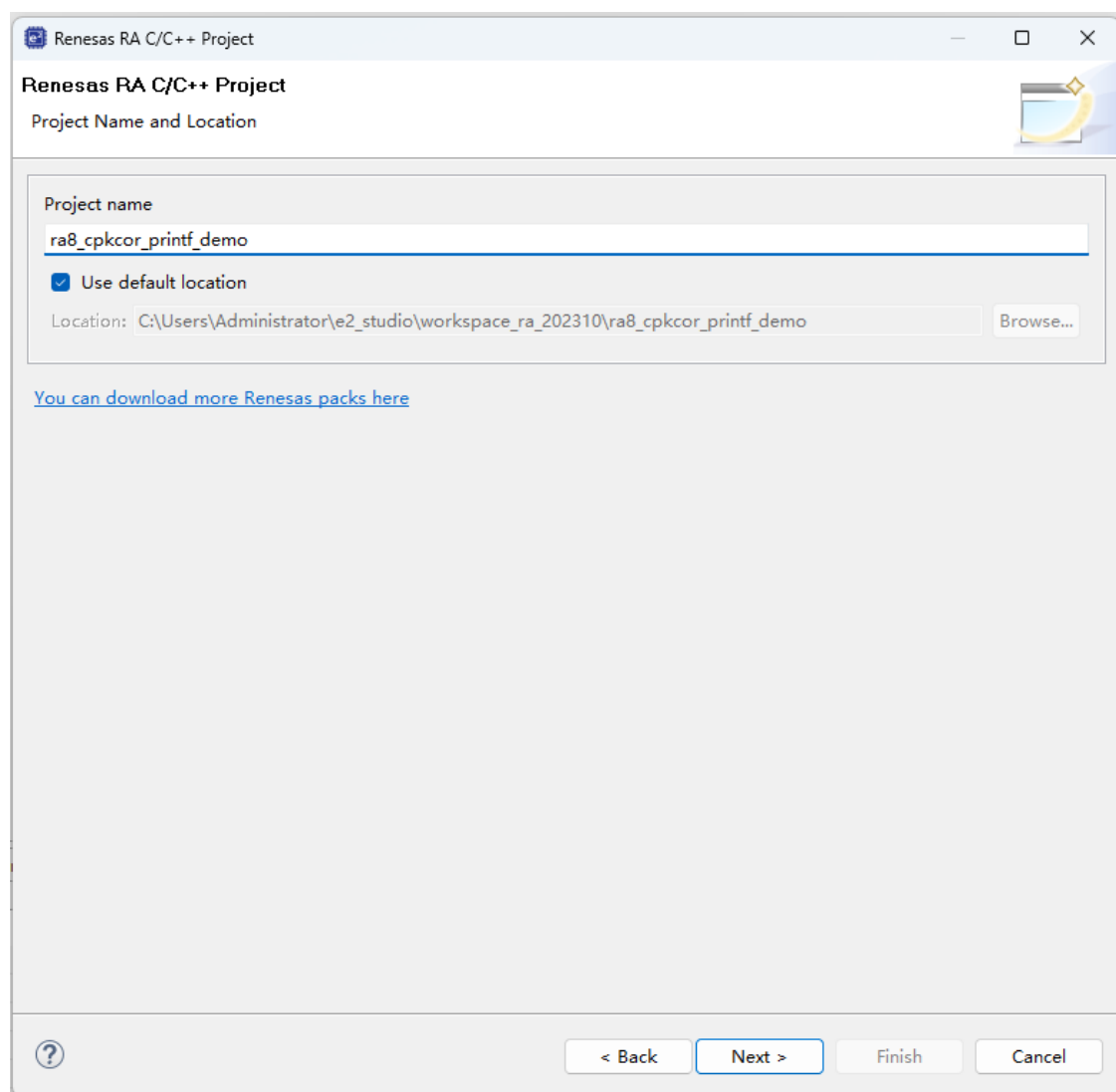
步骤

- 2.1 运行 “C:\Renesas\RA\e2studio_v2023-10_fsp_v5.1.0\eclipse\e2studio.exe”。打开 e2studio 2023-10。使用 Alt+Shift+N 组合键来创建新的 C/C++工程。选择 Renesas RA C/C++ Project。



2.2

创建“ra8_cpkcor_printf_demo”工程，运行“next”。



2.3

按照下图配置工程，然后运行“next”。

Device Selection:

- FSP Version: 5.1.0
- Board: CPKCOR-RA8D1B核心板

Toolchains:

- LLVM Embedded Toolchain for Arm: 17.0.1

The screenshot shows the 'Renesas RA C/C++ Project' configuration window. The title bar is blue with the Renesas logo and window controls. The main area is titled 'Renesas RA C/C++ Project' and 'Device and Tools Selection'. It is divided into several sections:

- Device Selection:**
 - FSP Version: 5.1.0 (dropdown)
 - Board: CPKCOR-RA8D1B核心板 (dropdown)
 - Device: R7FA8D1BHECBD (text input)
 - Core: CM85 (dropdown)
 - Language: ☒ C ☐ C++
- Board Description:**

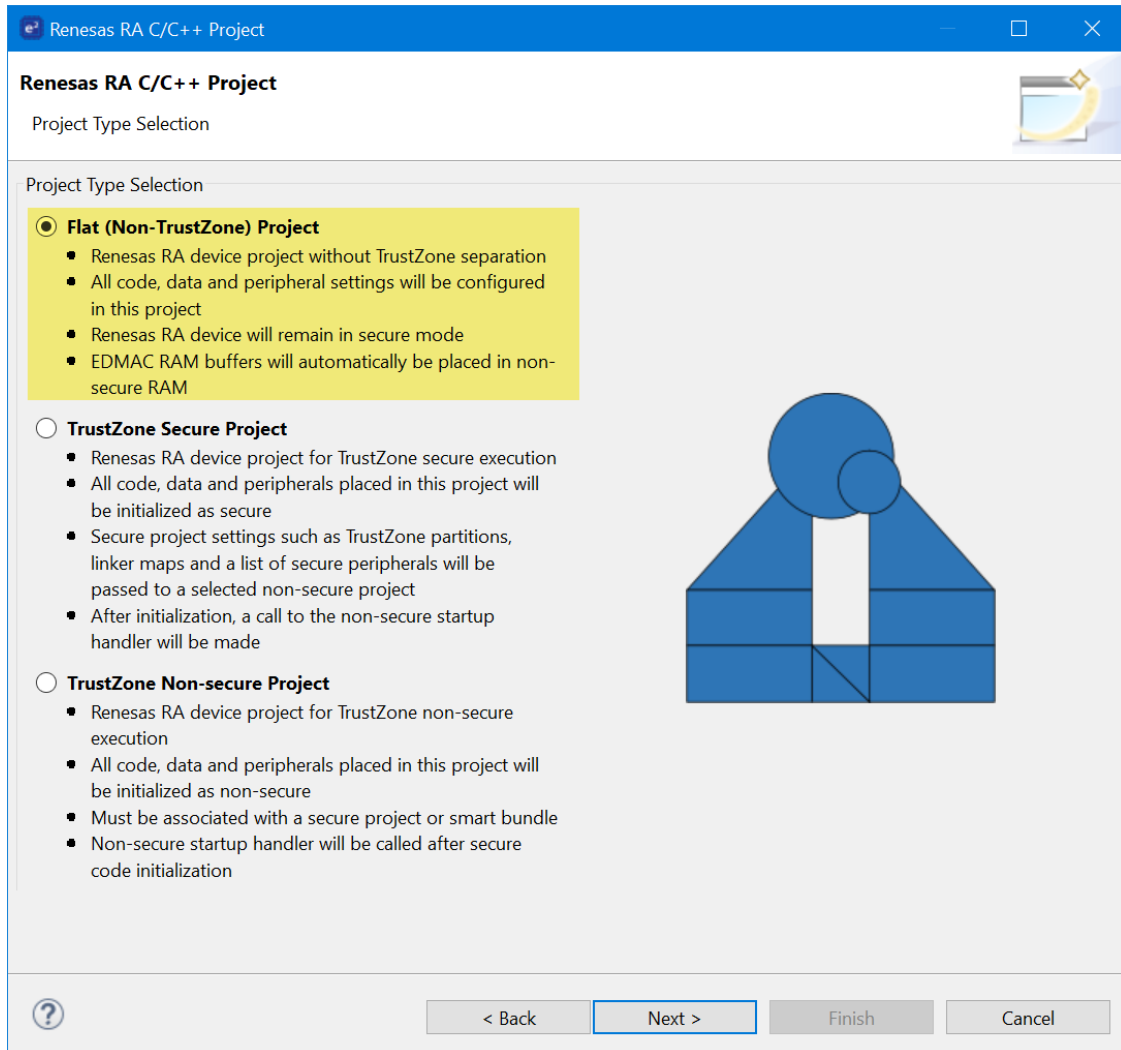
China promotion kit for RA8D1 MCU Group
Visit TBD to get kit user's manual, quick start guide, errata, design package, example projects, etc.
- Device Details:**

TrustZone	Yes
Pins	224
Processor	Cortex-M85
- Toolchains:**
 - GNU ARM Embedded
 - LLVM Embedded Toolchain for Arm (highlighted)
 - ARM Compiler 6.19
 - 17.0.1 (dropdown)
- Debugger:**
 - J-Link ARM (dropdown)

At the bottom, there is a help icon, a '< Back' button, a 'Next >' button (highlighted), a 'Finish' button, and a 'Cancel' button.

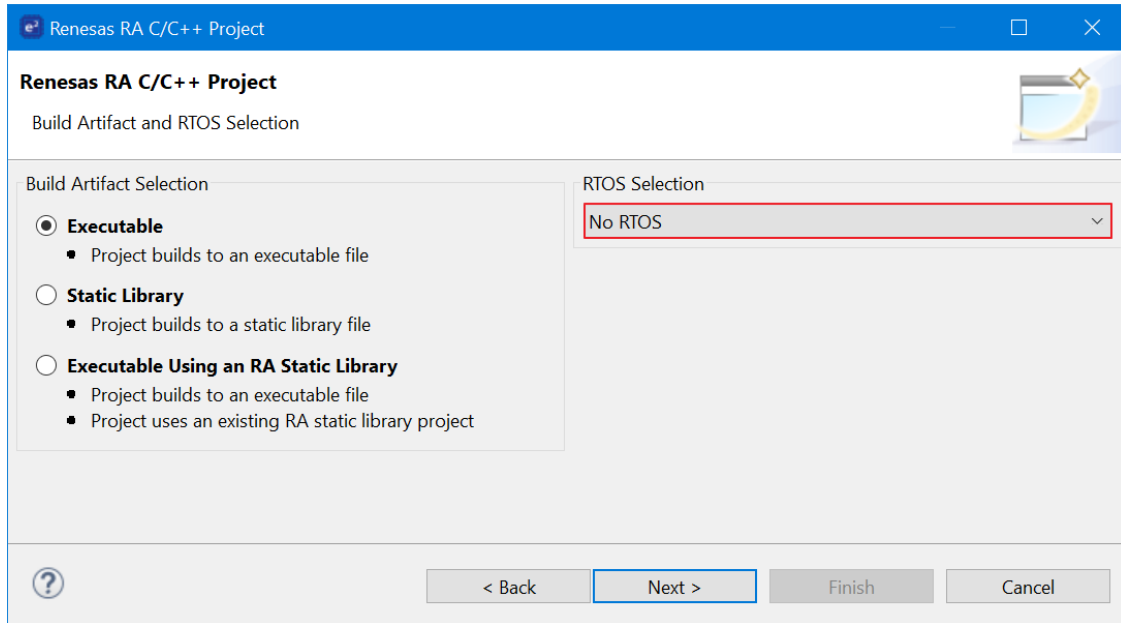
2.4

本次示例中，我们只关注RA8的裸机系统学习，选择Flat (Non-TrustZone) Project。



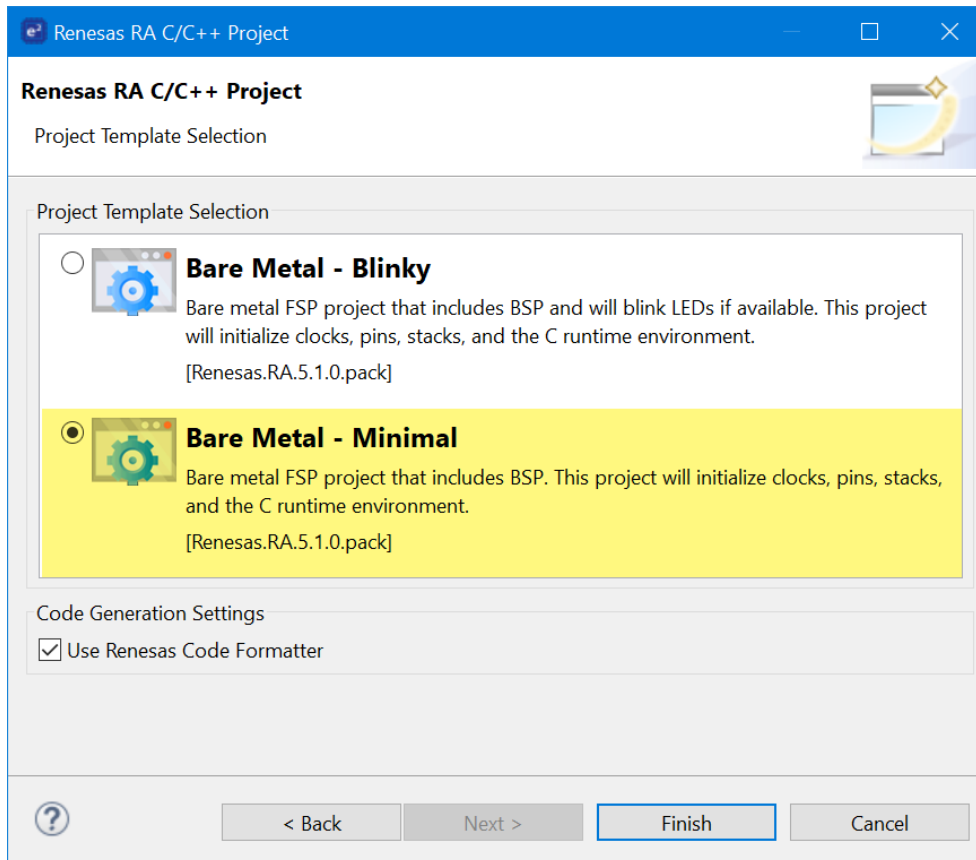
2.5

选择 “No RTOS”，然后运行 “Next”。



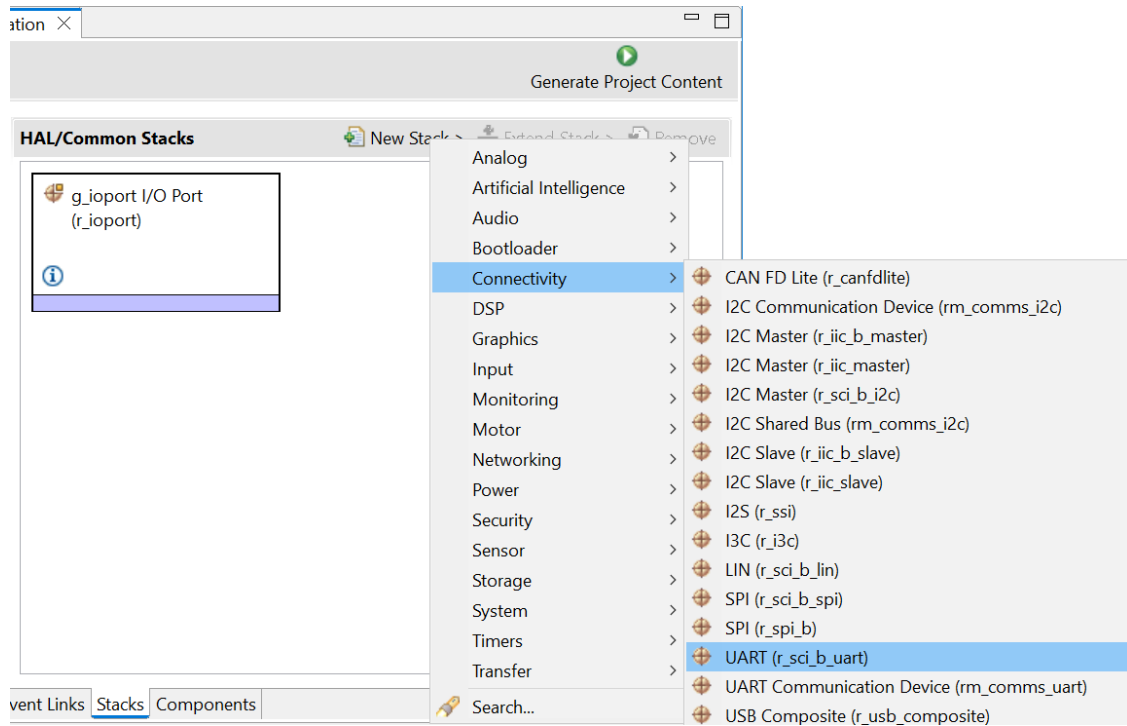
2.6

选择 “Bare Metal – Minimal”，然后运行 “Finish”。



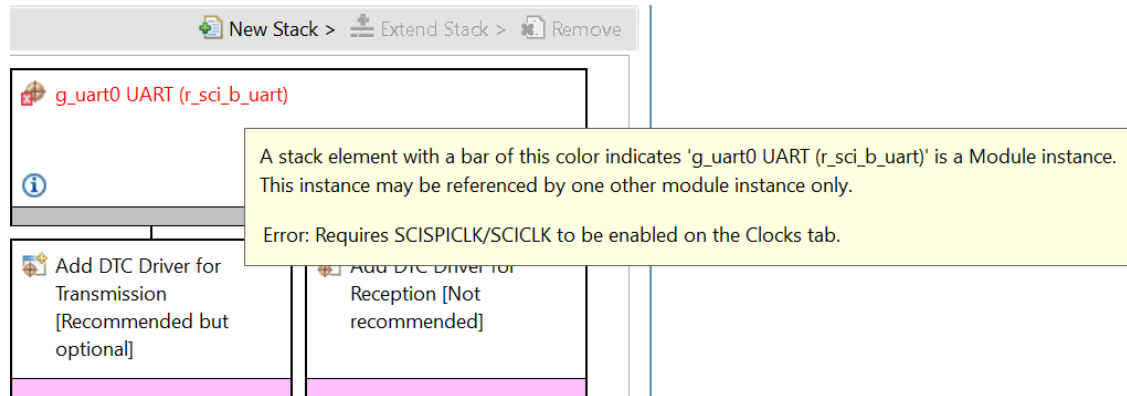
2.7

工程创建后，默认会打开configuration.xml文件。切换到Stack后，选择New Stack > Connectivity>UART(r_sci_b_uart)。



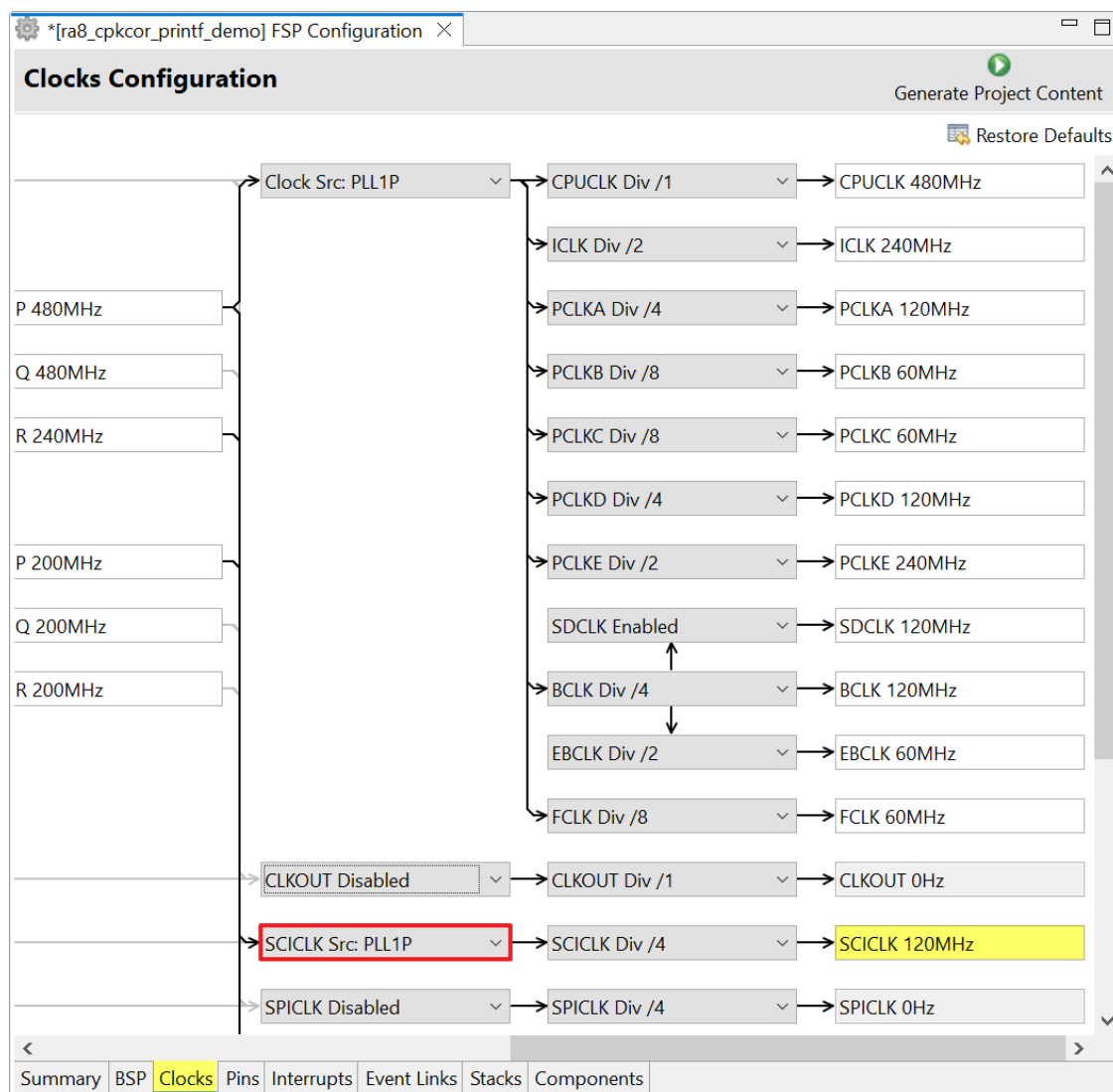
2.8

此时会出现一个错误信息，是因为没有使能SCICLK造成的。



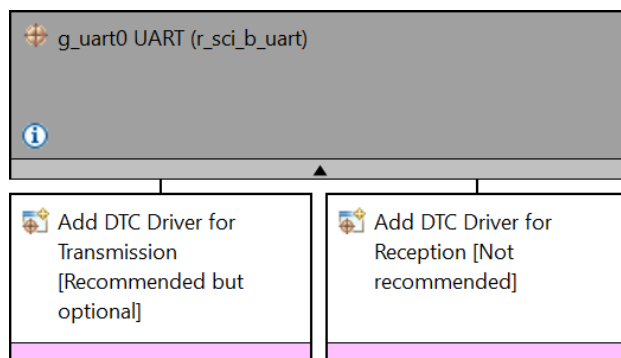
2.9

切换到Clock并使能SCICLK。SCICLK时钟源是PLL1P。确保SCICLK为120MHz后，保存配置。



2.10

切换到Stack并确定之前错误已经被解决。



2.11

在“Properties”中选择 g_uart0 UART (r_sci_b_uart)。

更新配置按照下面所示。

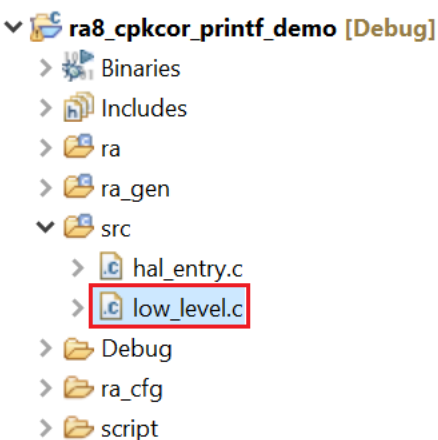
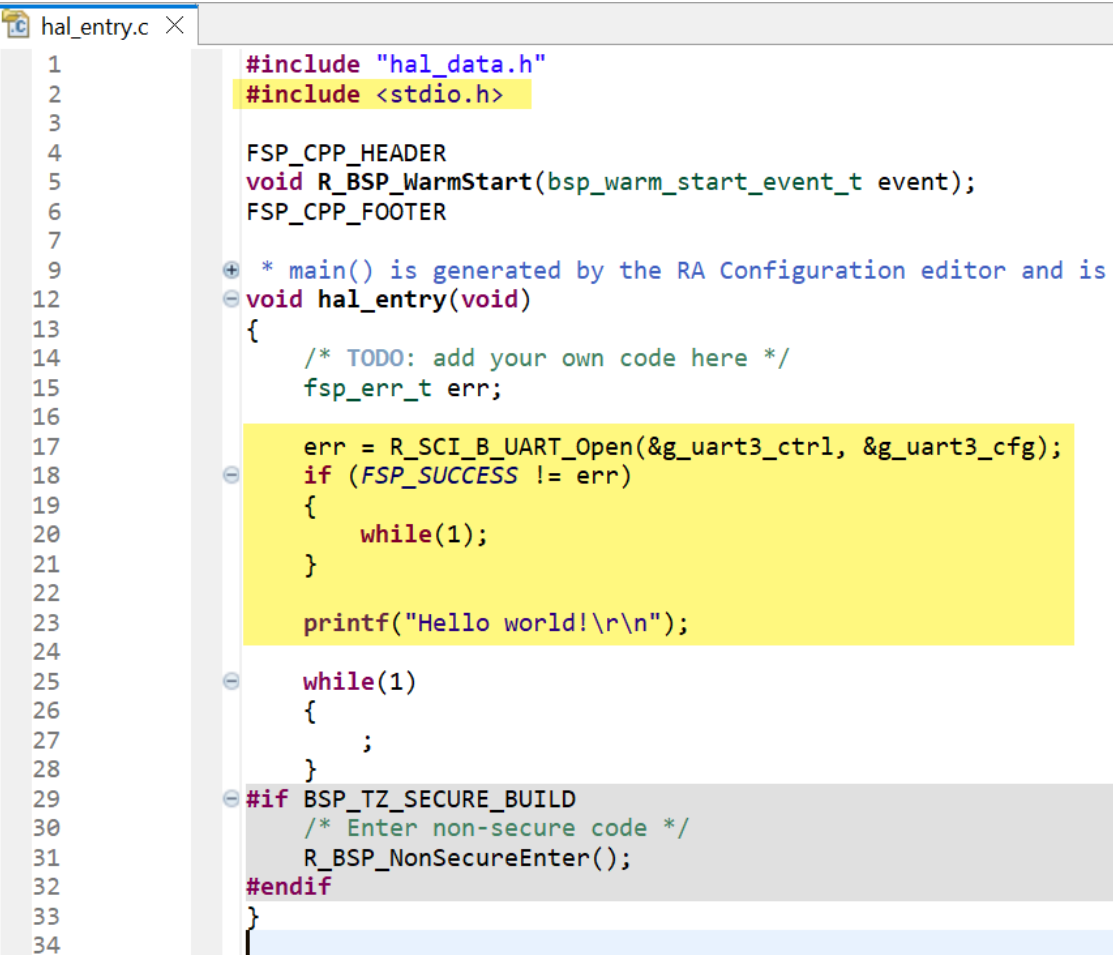
- General
 - Name: g_uart3
 - Channel: 3
- Interrupts
 - Callback: user_uart3_callback

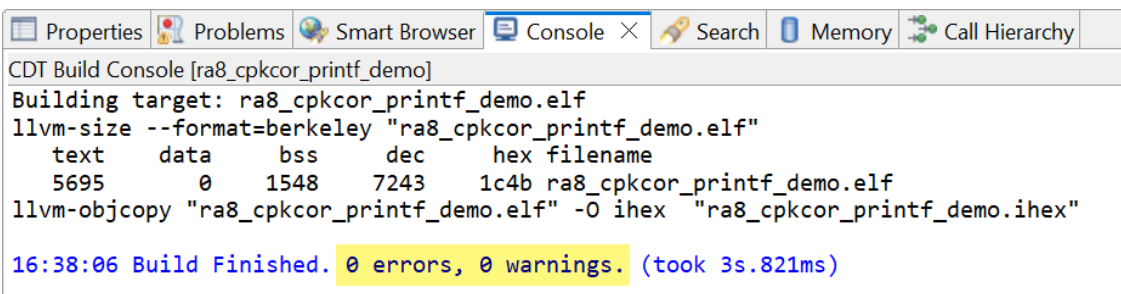
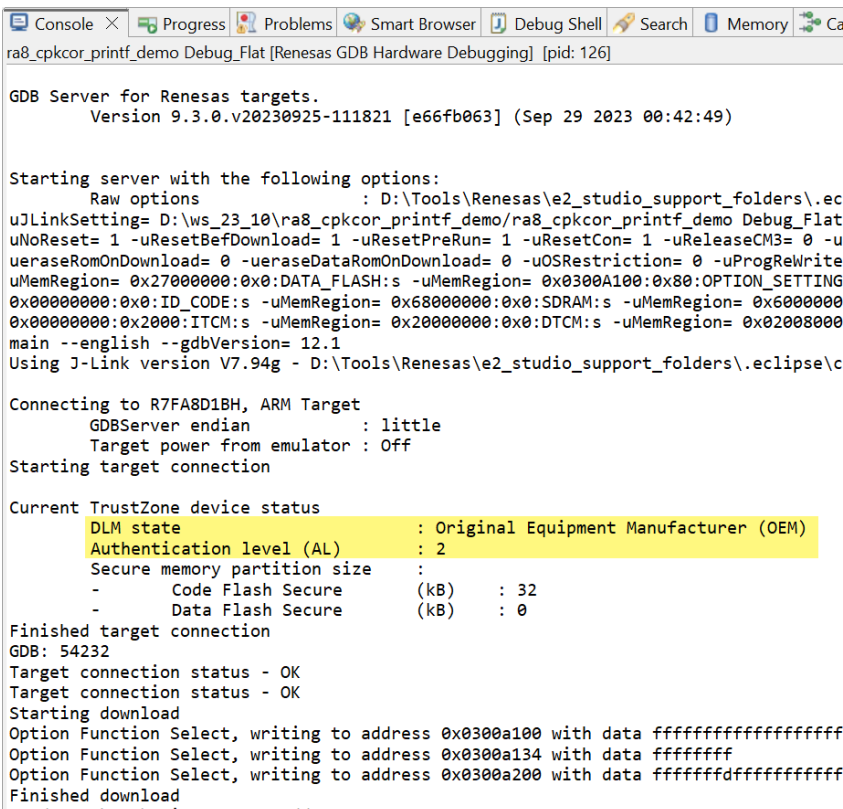
检查引脚配置：

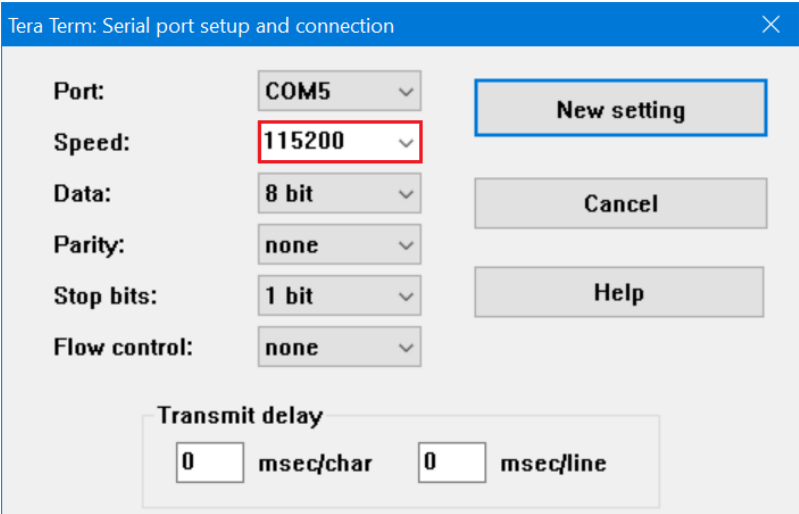
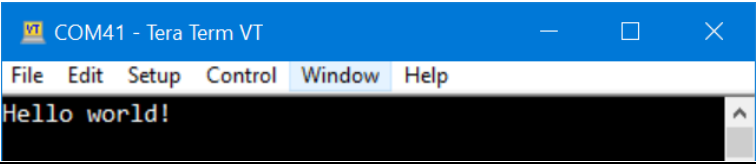
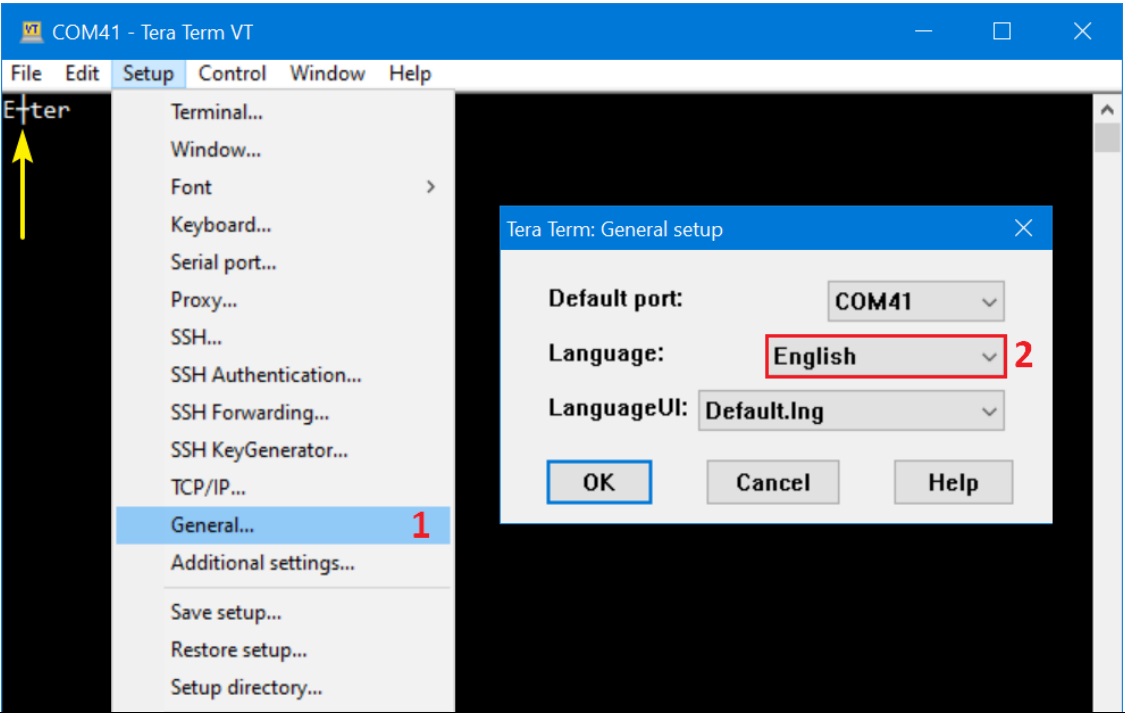
- RXD3 P408
- TXD3 P409

The screenshot shows the 'Properties' window for the 'g_uart0 UART (r_sci_b_uart)' component. The 'Settings' tab is active, and the 'General' section is expanded. The 'Name' is set to 'g_uart3' and the 'Channel' is set to '3'. The 'Data Bits' are set to '8bits', 'Parity' is 'None', and 'Stop Bits' are '1bit'. The 'Interrupts' section is also expanded, showing the 'Callback' set to 'user_uart3_callback'. The 'Pins' section is expanded, showing 'RXD3' connected to 'P408' and 'TXD3' connected to 'P409'.

Property	Value
Common	
Module g_uart3 UART (r_sci_b_uart)	
General	
Name	g_uart3
Channel	3
Data Bits	8bits
Parity	None
Stop Bits	1bit
Baud	
Flow Control	
Extra	
Interrupts	
Callback	user_uart3_callback
Receive Interrupt Priority	Priority 12
Transmit Data Empty Interrupt Priority	Priority 12
Transmit End Interrupt Priority	Priority 12
Error Interrupt Priority	Priority 12
Pins	
CTS3	None
CTS_RTS3	None
RXD3	P408
TXD3	P409

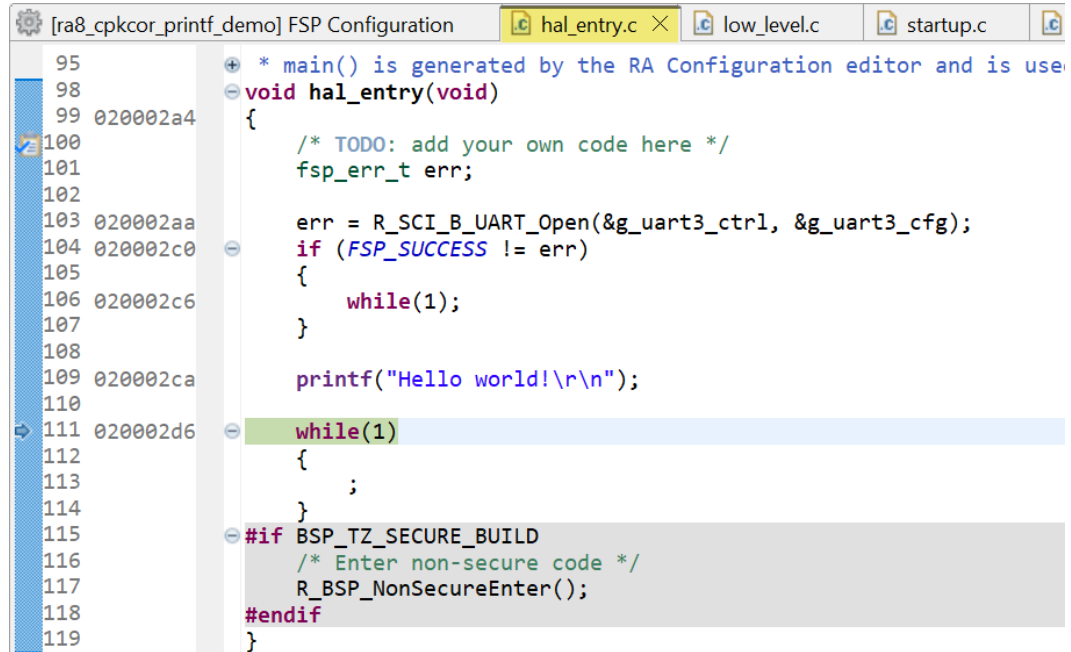
2.12	复制文件low_level.c到本工程的源文件夹。  ▼ ra8_cpkcor_printf_demo [Debug] > Binaries > Includes > ra > ra_gen ▼ src > hal_entry.c > low_level.c > Debug > ra_cfg > script
2.13	按照以下高亮的内容修改hal_entry.c文件。  <pre> 1 #include "hal_data.h" 2 #include <stdio.h> 3 4 FSP_CPP_HEADER 5 void R_BSP_WarmStart(bsp_warm_start_event_t event); 6 FSP_CPP_FOOTER 7 9 + * main() is generated by the RA Configuration editor and is 12 - void hal_entry(void) 13 { 14 /* TODO: add your own code here */ 15 fsp_err_t err; 16 17 err = R_SCI_B_UART_Open(&g_uart3_ctrl, &g_uart3_cfg); 18 if (FSP_SUCCESS != err) 19 { 20 while(1); 21 } 22 23 printf("Hello world!\r\n"); 24 25 while(1) 26 { 27 ; 28 } 29 - #if BSP_TZ_SECURE_BUILD 30 /* Enter non-secure code */ 31 R_BSP_NonSecureEnter(); 32 #endif 33 } 34 </pre>

2.14	点击图标  来编译工程。本工程经过编译后，应改是没有任何errors 或者 warnings。  <pre> CDT Build Console [ra8_cpkcor_printf_demo] Building target: ra8_cpkcor_printf_demo.elf llvm-size --format=berkeley "ra8_cpkcor_printf_demo.elf" text data bss dec hex filename 5695 0 1548 7243 1c4b ra8_cpkcor_printf_demo.elf llvm-objcopy "ra8_cpkcor_printf_demo.elf" -O ihex "ra8_cpkcor_printf_demo.ihex" 16:38:06 Build Finished. 0 errors, 0 warnings. (took 3s.821ms) </pre>
2.15	点击按钮  启动调试并检查控制台中的内容是否成功建立了连接。J-Link 版本信息和 DLM 状态会显示在 log 中。  <pre> Console X Progress Problems Smart Browser Debug Shell Search Memory Ca ra8_cpkcor_printf_demo Debug_Flat [Renesas GDB Hardware Debugging] [pid: 126] GDB Server for Renesas targets. Version 9.3.0.v20230925-111821 [e66fb063] (Sep 29 2023 00:42:49) Starting server with the following options: Raw options : D:\Tools\Renesas\e2_studio_support_folders\.ec uJLinkSetting= D:\ws_23_10\ra8_cpkcor_printf_demo\ra8_cpkcor_printf_demo Debug_Flat uNoReset= 1 -uResetBefDownload= 1 -uResetPreRun= 1 -uResetCon= 1 -uReleaseCM3= 0 -u ueraseRomOnDownload= 0 -ueraseDataRomOnDownload= 0 -uOSRestriction= 0 -uProgRewrite uMemRegion= 0x27000000:0x0:DATA_FLASH:s -uMemRegion= 0x0300A100:0x80:OPTION_SETTING 0x00000000:0x0:ID_CODE:s -uMemRegion= 0x68000000:0x0:SDRAM:s -uMemRegion= 0x60000000 0x00000000:0x2000:ITCM:s -uMemRegion= 0x20000000:0x0:DTCM:s -uMemRegion= 0x02008000 main --english --gdbVersion= 12.1 Using J-Link version V7.94g - D:\Tools\Renesas\e2_studio_support_folders\.eclipse\c Connecting to R7FA8D1BH, ARM Target GDBServer endian : little Target power from emulator : Off Starting target connection Current TrustZone device status DLM state : Original Equipment Manufacturer (OEM) Authentication level (AL) : 2 Secure memory partition size : - Code Flash Secure (kB) : 32 - Data Flash Secure (kB) : 0 Finished target connection GDB: 54232 Target connection status - OK Target connection status - OK Starting download Option Function Select, writing to address 0x0300a100 with data ffffffff Option Function Select, writing to address 0x0300a134 with data ffffffff Option Function Select, writing to address 0x0300a200 with data ffffffff Finished download Hardware breakpoint set at address 0x200031e </pre>

2.16	打开串口工具，将波特率设定为 115200。以下是 Tera Term 波特率的修改界面。 
2.17	点击图片  两次来运行代码。你会在 Tera Term 窗口中看到 “Hello world!”。 
2.18	如果你的窗口中显示的是乱码，请尝试修改语言设置来解决这个问题。比如，如果你的乱码是在 “E” 字母后面，请在通用安装中更新语言设置为 “English”。 

2.19

点击图标  来暂停代码运行。PC 指针将暂停在 `while(1)`。点击图标  来终止连接。



```

95  * main() is generated by the RA Configuration editor and is use
98  void hal_entry(void)
99  {
100  /* TODO: add your own code here */
101  fsp_err_t err;
102
103  err = R_SCI_B_UART_Open(&g_uart3_ctrl, &g_uart3_cfg);
104  if (FSP_SUCCESS != err)
105  {
106      while(1);
107  }
108
109  printf("Hello world!\r\n");
110
111  while(1)
112  {
113      ;
114  }
115  #if BSP_TZ_SECURE_BUILD
116  /* Enter non-secure code */
117  R_BSP_NonSecureEnter();
118  #endif
119  }
    
```

本节完

3 Helium intrinsic 程序调试

本节要点

在第二步的基础之上，调试 Helium intrinsic 程序。

步骤

3.1 按照上一步创建“ra8_cpkcor_printf_demo”的步骤，创建一个“ra8_cpkcor_helium_demo”的工程。需要注意的是，由于该工程会对图片进行处理，所以会使用到比较大的 RAM，需要事先进行堆栈设定。切换到 BSP，按照如下配置进行堆栈设置。

- stack 0x68000
- heap 0x1000

Board Support Package Configuration

Device Selection

FSP version: 5.1.0

Board: CPKCOR-RA8D1B核心板

Device: R7FA8D1BHECBD

Core: CM85

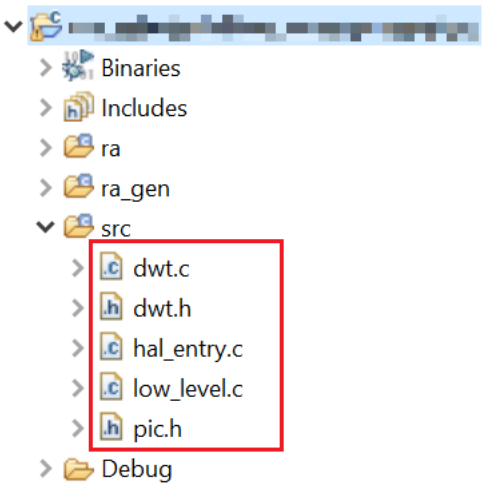
RTOS: No RTOS

Board Details
China promotion
Visit TBD to get

Summary **BSP** Clocks Pins Interrupts Event Links Stacks Components

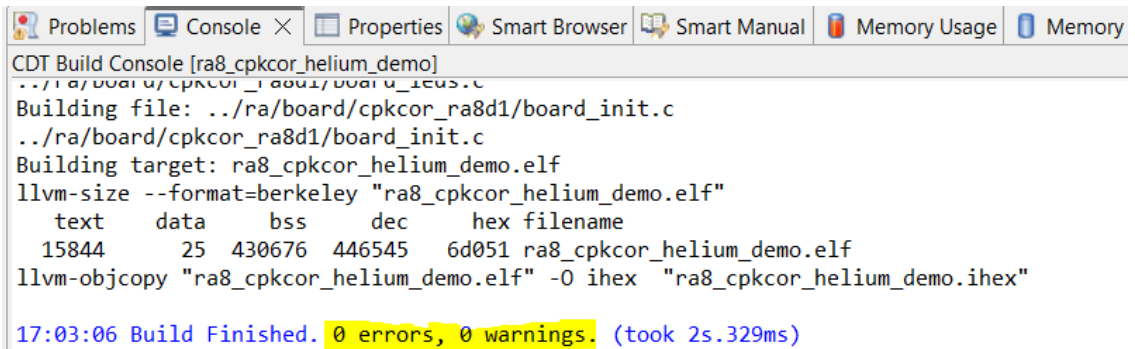
CPKCOR-RA8D1B核心板

Settings	Property	Value
	> OFS1 register settings	
	> OFS2 register settings	
	> Block Protection Settings (BPS)	
	> Permanent Block Protection Settings (PBPS)	
	> First Stage Bootloader (FSBL)	
	> Clocks	
	> Cache settings	
	Dual Bank Mode	Disabled
	Main Oscillator Wait Time	8163 cycles
	▼ RA Common	
	Main stack size (bytes)	0x68000
	Heap size (bytes)	0x1000
	MCU Vcc (mV)	3300
	Parameter checking	Disabled
	Assert Failures	Return FSP_ERR_ASSERTION
	Error Log	No Error Log

3.2	复制文件dwt.c, dwt.h, low_level.c, hal_entry.c, pic.h到本工程的源文件夹。提示冲突时，选择替换原来的hal_entry.c。  dwt.c和dwt.h是DWT相关的代码，DWT是（Data Watchpoint and Trace）的缩写，它是ARM cortex M中的一个32位的向上计数器，记录的是内核时钟运行个数，内核时钟跳动一次，该计数器就加1，精度非常高，用于系统调试和跟踪，也可以用来测试代码运行时间。 pic.h是一张240*160的32-bit RGBA8888图片数据。 
3.3	在 hal_entry.c 中的第 12~13 行，有两个宏定义。 <pre> 12 #define HELIUM_BASIC_DEMO 13 #define HELIUM_RGB_DEINTERLEAVE_DEMO </pre> HELIUM_BASIC_DEMO 是使能 helium 的基础例子的开关，它展示使用 helium 原语函数 (intrinsic)来进行加法，减法和乘法运算。 HELIUM_RGB_DEINTERLEAVE_DEMO 是使能 helium 进行图像 RGBA 数据色彩数据分离例子的开关，它展示了在色彩数据分离中，普通 c 代码实现和通过使用 helium 原语函数(intrinsic)的实现之间的效率对比。 注：HELIUM_BASIC_DEMO 的优先级比 HELIUM_RGB_DEINTERLEAVE_DEMO 高，如果这两个宏同时定义，hal_entry.c 中的第 15~17 行代码会自动进行 #undef HELIUM_RGB_DEINTERLEAVE_DEMO 操作。

3.4

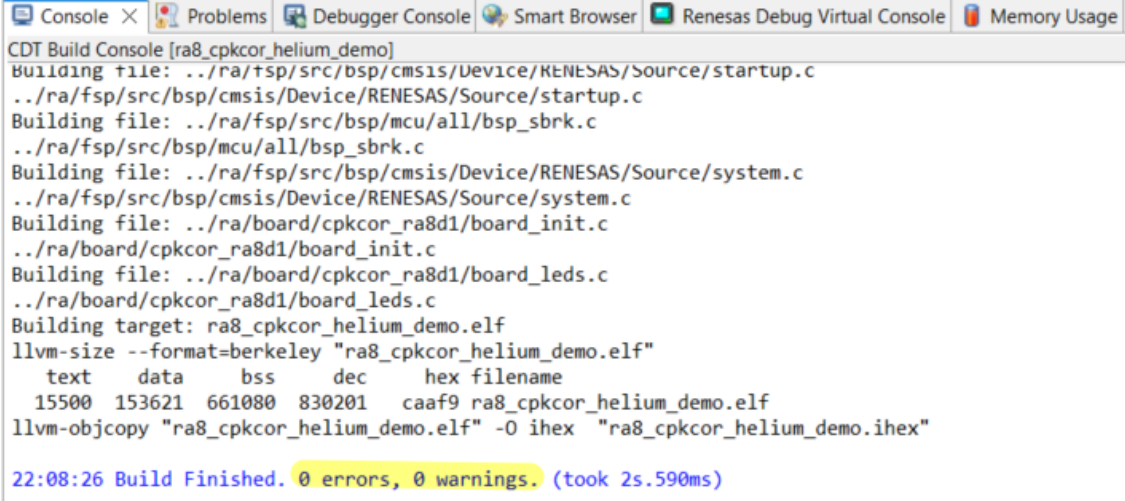
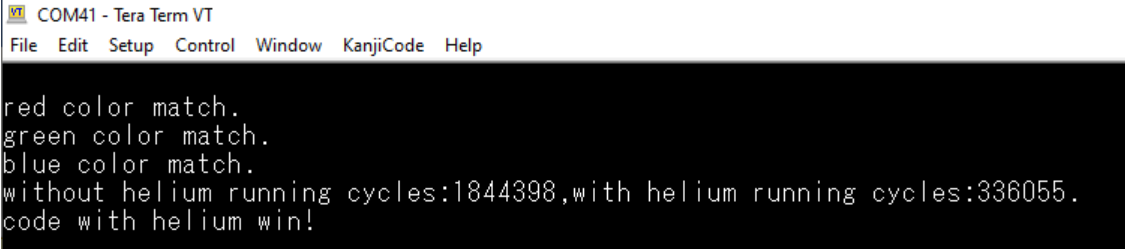
点击图标  来编译工程。本工程经过编译后，应改是没有任何 errors 或者 warnings。



```

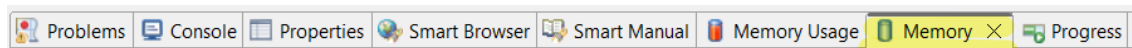
CDT Build Console [ra8_cpkcor_helium_demo]
../ra/board/cpkcor_ra8d1/board_init.c
Building file: ../ra/board/cpkcor_ra8d1/board_init.c
Building target: ra8_cpkcor_helium_demo.elf
llvm-size --format=berkeley "ra8_cpkcor_helium_demo.elf"
  text    data    bss     dec     hex filename
 15844     25  430676  446545  6d051 ra8_cpkcor_helium_demo.elf
llvm-objcopy "ra8_cpkcor_helium_demo.elf" -O ihex "ra8_cpkcor_helium_demo.ihex"

17:03:06 Build Finished. 0 errors, 0 warnings. (took 2s.329ms)
  
```

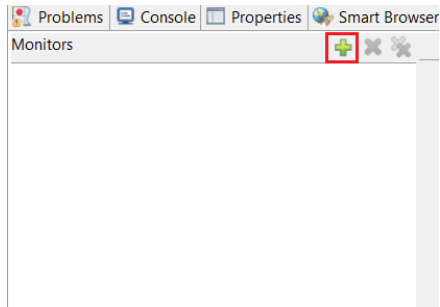

3.6	本示例中使用的原语函数(intrinsic) - uint8x16_t vld1q_u8(uint8_t const*base) 从内存中加载数据到uint8x16_t (16个uint8数据)类型矢量寄存器中。 - void vst1q_u8(uint8_t *base,uint8x16_t value) 将uint8x16_t (16个uint8数据)类型矢量寄存器中的数据写到内存中。 - uint8x16_t vaddq_u8(uint8x16_t a,uint8x16_t b) 将两个uint8x16_t (16个uint8数据)类型矢量寄存器中的数据相加，并返回计算结果。 - uint8x16_t vsubq_u8(uint8x16_t a,uint8x16_t b) 将两个uint8x16_t (16个uint8数据)类型矢量寄存器中的数据相减，并返回计算结果。 - uint32x4_t vld1q_u32(uint32_t const*base) 从内存中加载数据到uint32x4_t (4个uint32数据)类型矢量寄存器中。 - uint32x4_t vmulq_n_u32(uint32x4_t a,uint32_t b) 将两个uint32x4_t (4个uint32_t数据)类型矢量寄存器中的数据相乘，并返回计算结果。
3.7	注释掉 hal_entry.c 的第 12 行 “#define HELIUM_BASIC_DEMO”。再点击图标  来编译工程。本工程经过编译后，应改是没有任何 errors 或者 warnings。  <pre> CDT Build Console [ra8_cpkcor_helium_demo] Building file: ../ra/fsp/src/bsp/cmsis/Device/RENESAS/Source/startup.c ../ra/fsp/src/bsp/cmsis/Device/RENESAS/Source/startup.c Building file: ../ra/fsp/src/bsp/mcu/all/bsp_sbrk.c ../ra/fsp/src/bsp/mcu/all/bsp_sbrk.c Building file: ../ra/fsp/src/bsp/cmsis/Device/RENESAS/Source/system.c ../ra/fsp/src/bsp/cmsis/Device/RENESAS/Source/system.c Building file: ../ra/board/cpkcor_ra8d1/board_init.c ../ra/board/cpkcor_ra8d1/board_init.c Building file: ../ra/board/cpkcor_ra8d1/board_leds.c ../ra/board/cpkcor_ra8d1/board_leds.c Building target: ra8_cpkcor_helium_demo.elf llvm-size --format=berkeley "ra8_cpkcor_helium_demo.elf" text data bss dec hex filename 15500 153621 661080 830201 caaf9 ra8_cpkcor_helium_demo.elf llvm-objcopy "ra8_cpkcor_helium_demo.elf" -O ihex "ra8_cpkcor_helium_demo.ihex" 22:08:26 Build Finished. 0 errors, 0 warnings. (took 2s.590ms) </pre>
3.8	点击按钮  启动调试并检查控制台中的内容是否成功建立了连接。点击图片  两次来运行代码。你会在 Tera Term 窗口中看到运行结果。  <pre> COM41 - Tera Term VT File Edit Setup Control Window KanjiCode Help red color match. green color match. blue color match. without helium running cycles:1844398,with helium running cycles:336055. code with helium win! </pre> 可以发现，使用了 helium 技术的代码比普通的 c 代码运行要快的多。

3.9

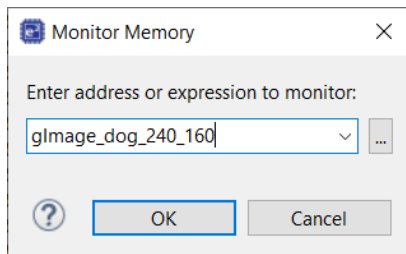
在 memory 中查看待处理的图像数据和处理后的图像数据。



点击图标 , 如下图所示



在弹出的窗口中输入 glImage_dog_240_160, 点击“OK”。

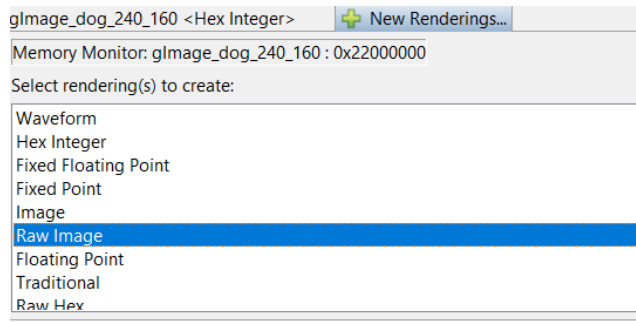


显示如下界面, 再点击  New Renderings...

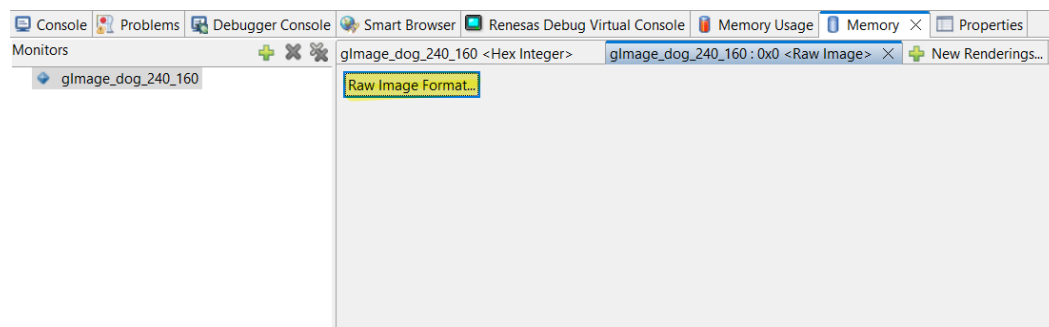
Monitors					glImage_dog_240_160 : 0x22000000 <Hex Integer>					New Renderings...				
glImage_dog_240_160					Address	0 - 3	4 - 7	8 - B	C - F					
					0000000022000000	0084818A	007A7780	00716E77	006D6A73					
					0000000022000010	0067646D	005A5760	004A4750	00403D46					
					0000000022000020	00211E27	0016131C	0017141D	00181821					
					0000000022000030	0016131C	0018151E	001A1720	00110E17					
					0000000022000040	0016131C	0016131C	0016131C	0016131C					
					0000000022000050	0016131C	0016131C	0016131C	0016131C					
					0000000022000060	00161417	00161417	00161417	00171518					
					0000000022000070	00171518	00171518	00181619	00181619					
					0000000022000080	0017141B	0017141B	0018151C	0018151C					

3.10

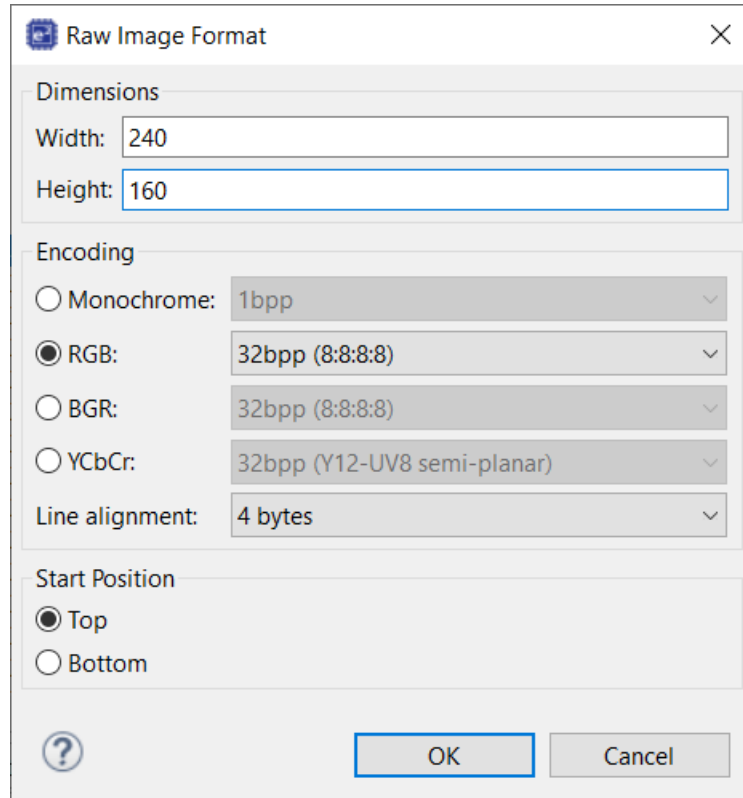
选择 Raw Image。



再选择 Raw Image Format。

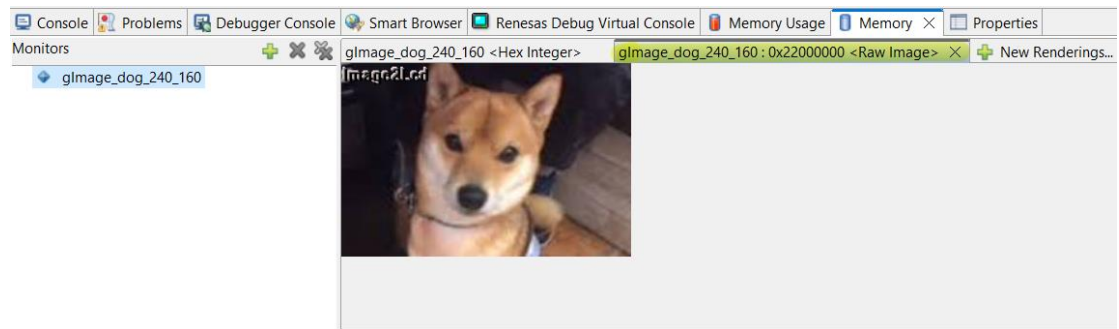


填入图片的格式，宽为 240，高为 160，编码为 RGB 32bpp(8:8:8:8)。点击 “OK”。

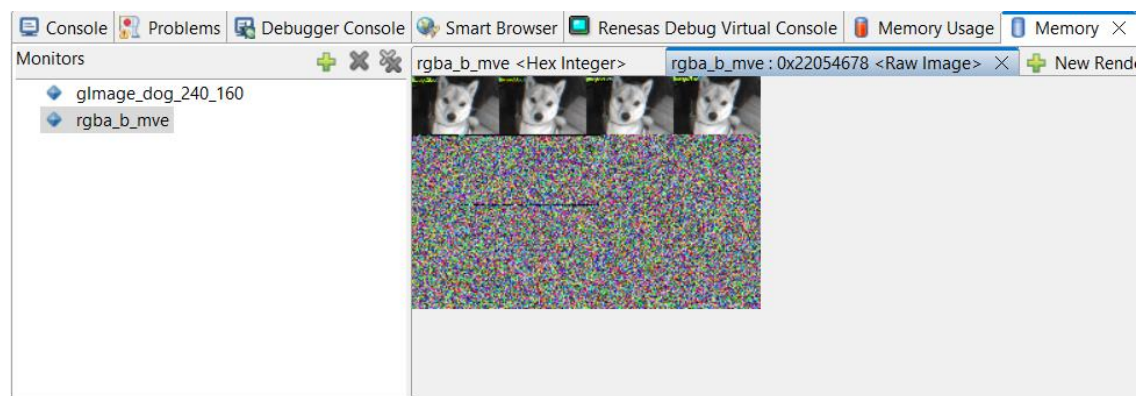


3.11

设置完成后，会在 glImage_dog_240_160:0x22000000<Raw Image>处显示彩色图片。

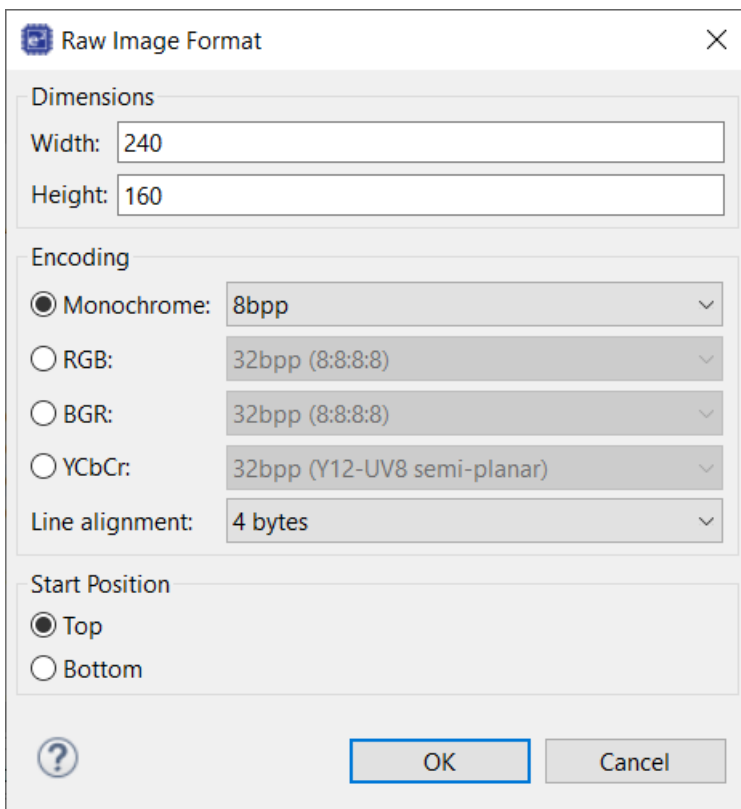
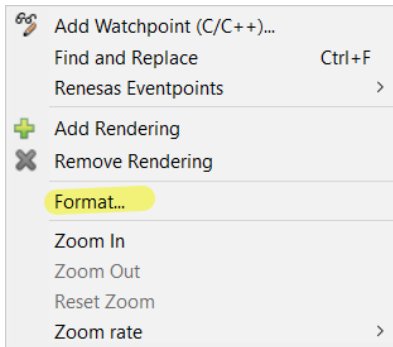


使用同样的方法，添加 rgba_b_mve 到 memory 中，会发现显示的图片不正常。



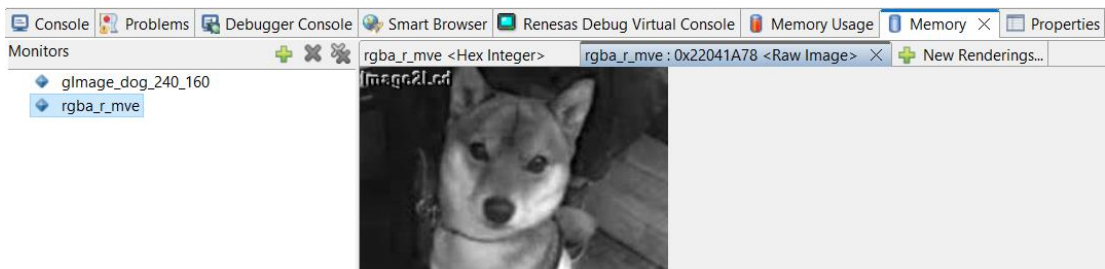
3.12

在图片上鼠标右键选择“Format” 填入参数宽为 240, 高为 160, 编码为 Monochrome 8bpp。点击“OK”。



3.13

这个时候，图片就显示正常了。因为现在这个数据是单一色彩通道的数据，所以，就是一个 8 位的灰度图。



3.14

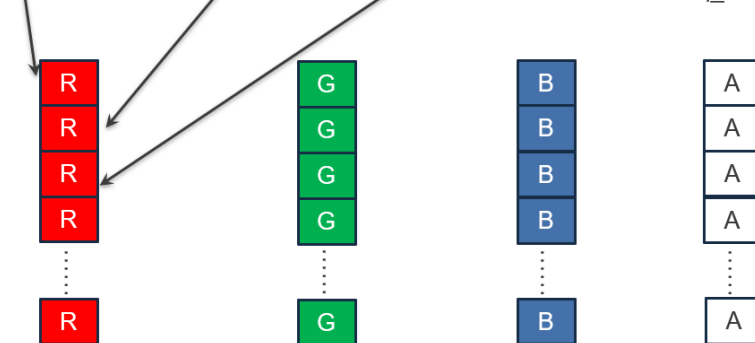
实验中的图片是 RGBA 格式，通过使用 Helium 的原语函数(intrinsic)，实现 RGBA 色彩数据分离。

原语函数	含义	作用
vld4q(...)	通过加载 4*16 字节内存的连续区域返回 uint8x16x4_t 的函数。加载的每个字节都以交替模式放置在 4 个 uint8x16_t 数组之一中。	生成 LD4 指令，该指令将给定地址中的值以交替模式加载到 4 个 helium 寄存器中。具体作用参考下图。
vst1q_u8(...)	在给定地址存储 uint8x16_t 的函数。	存储整个 128 位数据的函数。

16*8*4 memory data

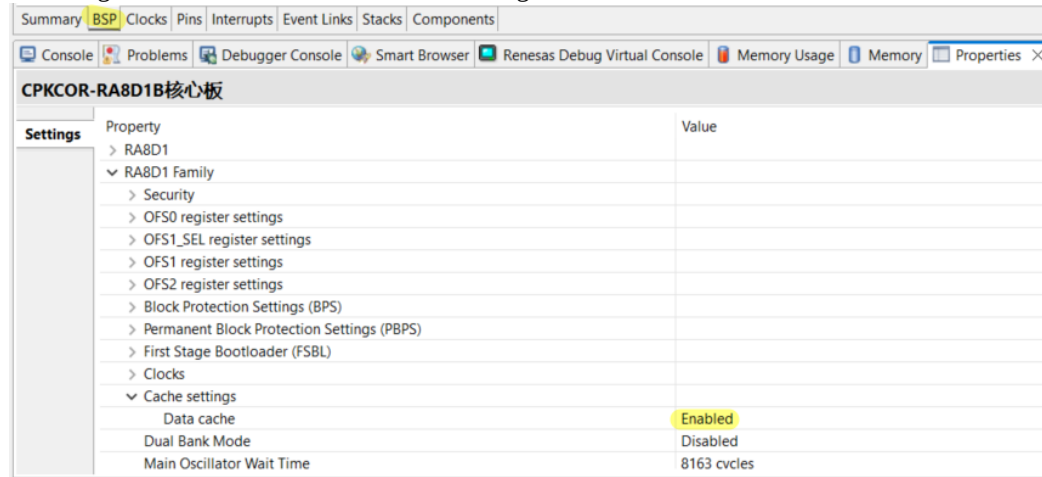


Vld4q_u8(...) deinterleave



3.15

把 configuration.xml 中 BSP 部分 Cache settings 中 Data cache 设置为 Enable，开启 Data Cache。



3.16

再次运行工程。会得到如下结果，可以发现 Data Cache 对程序的运行效率，有很大的提升。这是因为，Data Cache 在数据的访问速度上有提升。

```
COM41 - Tera Term VT
File Edit Setup Control Window KanjiCode Help

red color match.
green color match.
blue color match.
without helium running cycles:201196,with helium running cycles:106366.
code with helium win!
```

本节完

实验结束

感谢参与



扫码立即关注

瑞萨电子嵌入式小百科

新品发布|应用案例

开发经验|实践教学|送开发板